

Real-Time Examples of Interfaces in C#

In this article, I will discuss **Multiple Real-Time Examples of Interfaces in C#**. Please read our [Interface in C#](#) article, where we discussed the basic concepts of Interface. At the end of this article, you will understand the following Real-time Examples using the Interface in C#.

1. [What is an interface in C#?](#)
2. [Plugin System](#)
3. [Vehicles Management](#)
4. [Payment Gateway Integration](#)
5. [Database Operations](#)
6. [Logging System](#)
7. [Drawing Shape](#)
8. [User Authentication Methods](#)
9. [Advantages and Disadvantages of Interfaces in C#](#)
10. [When to use Interfaces in C#?](#)

What is an interface in C#?

An interface in C# is a type definition similar to a class but represents a contract rather than an implementation. It defines a set of method signatures, properties, events, or indexers without providing their implementations. Any class or structure that adopts this interface must provide a concrete implementation for each member defined by the interface. Here are some key points about interfaces in C#:

- **Contractual Obligation:** An interface acts as a contract. Any class or structure implementing the interface must fulfill this contract by providing implementations for all the interface members.
- **No Implementation:** An interface cannot contain any implementation. It can only have declarations.
- **Multiple Interfaces:** A class or struct can implement multiple interfaces. This provides a way to achieve multiple inheritance in C#, which doesn't support multiple inheritance for classes.
- **No Access Modifiers:** Interface members cannot have access modifiers like public, private, or protected. By default, they are public, but you don't specify the modifier.
- **Properties, Indexers, and Events:** Apart from methods, interfaces can also define properties, indexers, and events.

Real-Time Example of Interface in C#: Plugin System

Consider a plugin system for a chat application where multiple message processors can be plugged in. For example, one processor might convert text to uppercase, another might censor certain words, and another might translate the message into a different language. Let us implement this example using Interface in C#:

```
using System;
using System.Collections.Generic;
namespace InterfaceCSharp
{
    //Step 1: Define the IMessageProcessor interface.
    public interface IMessageProcessor
    {
        string ProcessMessage(string input);
    }
}
```

```

}
//Step 2: Create some implementations of the interface.
// UpperCaseProcessor.cs
public class UpperCaseProcessor : IMessageProcessor
{
    public string ProcessMessage(string input)
    {
        return input.ToUpper();
    }
}
// CensorshipProcessor.cs
public class CensorshipProcessor : IMessageProcessor
{
    private string[] forbiddenWords = { "badword1", "badword2" }; // Exar
censored words
    public string ProcessMessage(string input)
    {
        foreach (var word in forbiddenWords)
        {
            input = input.Replace(word, "****");
        }
        return input;
    }
}
//Step 3: In your chat application, implement the use of these processors.
public class ChatApp
{
    private List<IMessageProcessor> messageProcessors = new List<IMessageP
());
    public ChatApp()
    {
        // Add processors to the chat application
        messageProcessors.Add(new UpperCaseProcessor());
        messageProcessors.Add(new CensorshipProcessor());
    }
    public void SendMessage(string message)
    {
        foreach (var processor in messageProcessors)
        {
            message = processor.ProcessMessage(message);
        }
        Console.WriteLine("Sending Message: " + message);
        // Here, you'd typically send the message to the server or another
    }
}
//Step 4: Testing the application.
class Program
{
    static void Main(string[] args)
    {
        ChatApp chat = new ChatApp();
    }
}

```

```

        chat.SendMessage("Hello! Please avoid using badword1 in our chat."
        // Outputs: "Sending Message: HELLO! PLEASE AVOID USING **** IN OL
        Console.ReadKey());
    }
}
}

```

This is a simple example, but the idea is that by using interfaces, we can easily extend the chat application's functionality by plugging in different message processors. This promotes the Open/Closed principle (one of the SOLID principles), where software entities should be open for extension but closed for modification. In this case, you can add more processors without altering existing code.

Real-Time Example of Interface in C#: Vehicles

Imagine you have different types of vehicles like cars, boats, and airplanes. Each of these vehicles can move, but how they move differs. Create a simple program where each vehicle type displays its mode of movement. Let us implement this example using Interface in C#:

```

using System;
using System.Collections.Generic;
namespace InterfaceCSharp
{
    //Step 1: Define the IMovable interface.
    public interface IMovable
    {
        void Move();
    }
    //Step 2: Implement the interface for different types of vehicles.
    // Car.cs
    public class Car : IMovable
    {
        public void Move()
        {
            Console.WriteLine("The car drives on the road.");
        }
    }
    // Boat.cs
    public class Boat : IMovable
    {
        public void Move()
        {
            Console.WriteLine("The boat sails on the water.");
        }
    }
    // Airplane.cs
    public class Airplane : IMovable
    {
        public void Move()
        {
            Console.WriteLine("The airplane flies in the sky.");
        }
    }
}

```

```

    }
}

//Step 4: Testing the application.
class Program
{
    static void Main(string[] args)
    {
        List<IMovable> vehicles = new List<IMovable>
        {
            new Car(),
            new Boat(),
            new Airplane()
        };
        foreach (var vehicle in vehicles)
        {
            vehicle.Move();
        }
        Console.ReadKey();
    }
}
}

```

In this real-time example, the interface IMovable acts as a contract, ensuring that every vehicle type that implements it will have a Move method. This abstraction allows us to handle different types of vehicles uniformly, demonstrated in the foreach loop in the main program.

With this pattern, if we want to introduce more vehicle types (e.g., a bicycle, a skateboard, or a rocket), we create a new class for each and implement the IMovable interface. Our main program remains unchanged and functions correctly for any new vehicle type. When you run the above code, you will get the following output:

```

The car drives on the road.
The boat sails on the water.
The airplane flies in the sky.

```

Real-Time Example of Interface in C#: Payment Gateway Integration

This is a typical scenario in many applications, where you might want to support multiple payment providers (like PayPal, Stripe, and others). In this context, interfaces become invaluable because they allow us to define a consistent way of processing payments, regardless of the underlying provider. Let us implement this example using Interface in C#:

```

using System;
namespace InterfaceCSharp
{
    //Step 1: Define the IPaymentGateway interface.
    public interface IPaymentGateway
    {

```

```

        bool ProcessPayment(decimal amount);
    }
    //Step 2: Implement the interface for different payment providers.
    // PayPalPaymentGateway.cs
    public class PayPalPaymentGateway : IPaymentGateway
    {
        public bool ProcessPayment(decimal amount)
        {
            // Call PayPal's API to process the payment
            Console.WriteLine($"Processing ${amount} payment using PayPal...");
            return true; // assume success for the sake of this example
        }
    }
    // StripePaymentGateway.cs
    public class StripePaymentGateway : IPaymentGateway
    {
        public bool ProcessPayment(decimal amount)
        {
            // Call Stripe's API to process the payment
            Console.WriteLine($"Processing ${amount} payment using Stripe...");
            return true; // assume success for this example
        }
    }
    //Step 3: Use the implementations in a shopping cart scenario.
    public class ShoppingCart
    {
        private IPaymentGateway _paymentGateway;
        public ShoppingCart(IPaymentGateway paymentGateway)
        {
            _paymentGateway = paymentGateway;
        }
        public void Checkout(decimal amount)
        {
            if (_paymentGateway.ProcessPayment(amount))
            {
                Console.WriteLine("Payment was successful!");
            }
            else
            {
                Console.WriteLine("Payment failed. Please try again.");
            }
        }
    }
    //Step 4: Testing the application.
    class Program
    {
        static void Main(string[] args)
        {
            // Customer selects PayPal as their preferred payment method
            ShoppingCart cart = new ShoppingCart(new PayPalPaymentGateway());
            cart.Checkout(100.00M);
        }
    }

```

```

        // Another customer selects Stripe
        cart = new ShoppingCart(new StripePaymentGateway());
        cart.Checkout(200.00M);
        Console.ReadKey();
    }
}
}

```

In this real-time example, the `IPaymentGateway` interface abstracts the specifics of processing payments. The `ShoppingCart` class doesn't need to know the details of each payment provider. It is called `ProcessPayment`, and the correct payment provider takes care of the rest.

This architecture makes it easy to add more payment providers in the future. When you want to add a new one, implement the `IPaymentGateway` interface, and the rest of the system can remain unchanged. When you run the above code, you will get the following output:

```

Processing $100.00 payment using PayPal...
Payment was successful!
Processing $200.00 payment using Stripe...
Payment was successful!

```

Real-Time Example of Interface in C#: Database Operations

Suppose we're working on an application where we need to support interactions with different types of databases. By using interfaces, we can make our application flexible, extensible, and decoupled from the specifics of each database. Let us implement this example using Interface in C#:

```

using System;
namespace InterfaceCSharp
{
    //Step 1: Define the IDatabase interface.
    public interface IDatabase
    {
        void Connect();
        void Insert(string data);
        void Disconnect();
    }
    //Step 2: Implement the interface for different databases.
    // SQLiteDatabase.cs
    public class SQLiteDatabase : IDatabase
    {
        public void Connect()
        {
            Console.WriteLine("Connected to SQL Database.");
        }
        public void Insert(string data)
        {
            Console.WriteLine($"Inserted '{data}' into SQL Database.");
        }
    }
}

```

```

        public void Disconnect()
        {
            Console.WriteLine("Disconnected from SQL Database.");
        }
    }
// NoSqlDatabase.cs
public class NoSqlDatabase : IDatabase
{
    public void Connect()
    {
        Console.WriteLine("Connected to NoSQL Database.");
    }
    public void Insert(string data)
    {
        Console.WriteLine($"Inserted '{data}' into NoSQL Database.");
    }
    public void Disconnect()
    {
        Console.WriteLine("Disconnected from NoSQL Database.");
    }
}
//Step 3: Use the implementations in an application.
public class DatabaseManager
{
    private IDatabase _database;
    public DatabaseManager(IDatabase database)
    {
        _database = database;
    }
    public void AddData(string data)
    {
        _database.Connect();
        _database.Insert(data);
        _database.Disconnect();
    }
}
//Step 4: Testing the application.
class Program
{
    static void Main(string[] args)
    {
        // Using SQL Database
        DatabaseManager dbManager = new DatabaseManager(new SqlDatabase());
        dbManager.AddData("SampleData1");
        // Using NoSQL Database
        dbManager = new DatabaseManager(new NoSqlDatabase());
        dbManager.AddData("SampleData2");
        Console.ReadKey();
    }
}
}

```

In this real-world example, the `IDatabase` interface provides a clear contract for database operations, ensuring consistency regardless of the underlying database system. As a result, our `DatabaseManager` class becomes adaptable to any database system that implements the `IDatabase` interface. This means we can easily support new database systems without major changes to our application. When you run the above code, you will get the following output:

```
Connected to SQL Database.
Inserted 'SampleData1' into SQL Database.
Disconnected from SQL Database.
Connected to NoSQL Database.
Inserted 'SampleData2' into NoSQL Database.
Disconnected from NoSQL Database.
```

Real-Time Example of Interface in C#: Logging System

Many applications use logging mechanisms to track operations, errors, or important actions. An application may need to log messages to different outputs like a console, a file, or a remote service. Using interfaces, we can define a generic way to log messages regardless of where they are logged. Let us implement this example using Interface in C#:

```
using System;
namespace InterfaceCSharp
{
    //Step 1: Define the ILogger interface.
    public interface ILogger
    {
        void LogMessage(string message);
    }
    //Step 2: Implement the interface for different logging mechanisms.
    // ConsoleLogger.cs
    public class ConsoleLogger : ILogger
    {
        public void LogMessage(string message)
        {
            Console.WriteLine($"Console: {message}");
        }
    }
    // FileLogger.cs
    public class FileLogger : ILogger
    {
        private string filePath;
        public FileLogger(string filePath)
        {
            this.filePath = filePath;
        }
        public void LogMessage(string message)
        {
            System.IO.File.AppendAllText(filePath, message + Environment.NewLine);
        }
    }
}
```

```

}
//Step 3: Use the implementations in an application.
public class Application
{
    private ILogger _logger;
    public Application(ILogger logger)
    {
        _logger = logger;
    }
    public void Run()
    {
        // Example of a scenario where we want to log a message
        _logger.LogMessage("Application started.");
        // More code and operations...
        _logger.LogMessage("Application ended.");
    }
}
//Step 4: Testing the application.
class Program
{
    static void Main(string[] args)
    {
        // Log to console
        Application appWithConsoleLogging = new Application(new ConsoleLog
        appWithConsoleLogging.Run();
        // Log to file
        Application appWithFileLogging = new Application(new FileLogger("a
        appWithFileLogging.Run();
        Console.ReadKey();
    }
}
}

```

Additionally, the messages would be logged to an “app.log” file when the application is run with the FileLogger. In this example, the interface ILogger provides a unified way to log messages. Different logging mechanisms can be easily plugged into our application without changing the core code. This promotes separation of concerns and makes the codebase easily extensible to support new logging mechanisms in the future.

Real-Time Example of Interface in C#: Drawing Shape

Imagine you’re developing a graphics application where different shapes must be drawn on the screen. Each shape knows how to draw itself, but the way they are drawn is different. Create a simple program where each shape type can be drawn. Let us implement this example using Interface in C#:

```

using System;
using System.Collections.Generic;
namespace InterfaceCSharp
{
    //Step 1: Define the IDrawable interface.

```

```

public interface IDrawable
{
    void Draw();
}
//Step 2: Implement the interface for different shapes.
// Circle.cs
public class Circle : IDrawable
{
    public void Draw()
    {
        Console.WriteLine("Drawing a circle.");
    }
}
// Rectangle.cs
public class Rectangle : IDrawable
{
    public void Draw()
    {
        Console.WriteLine("Drawing a rectangle.");
    }
}
// Triangle.cs
public class Triangle : IDrawable
{
    public void Draw()
    {
        Console.WriteLine("Drawing a triangle.");
    }
}

//Step 4: Testing the application.
class Program
{
    static void Main(string[] args)
    {
        List<IDrawable> shapes = new List<IDrawable>
        {
            new Circle(),
            new Rectangle(),
            new Triangle()
        };
        foreach (var shape in shapes)
        {
            shape.Draw();
        }
        Console.ReadKey();
    }
}

```

In this example, the `IDrawable` interface is a contract for all drawable objects in the application. By adhering to this contract, each shape knows how to draw itself. The main program (i.e., `GraphicsApp`) can now work with a diverse range of drawable objects without needing to be aware of the specifics of how each shape is drawn.

Using this pattern, if you wanted to introduce more shapes (like a pentagon, hexagon, etc.), you would create a new class for each shape and implement the `IDrawable` interface. The core drawing logic in the main program would remain unchanged, demonstrating the extensibility that interfaces can bring to a system.

Real-Time Example of Interface in C#: User Authentication Methods

Imagine a scenario where an application allows users to authenticate using different methods, such as a password, fingerprint, or face recognition. Develop a mechanism that can support multiple authentication methods without changing the core authentication process. Let us implement this example using Interface in C#:

```
using System;
using System.Collections.Generic;
namespace InterfaceCSharp
{
    //Step 1: Define the IAuthenticator interface.
    public interface IAuthenticator
    {
        bool Authenticate();
    }
    //Step 2: Implement the interface for different authentication methods.
    // PasswordAuthenticator.cs
    public class PasswordAuthenticator : IAuthenticator
    {
        public bool Authenticate()
        {
            // Logic for password authentication
            Console.WriteLine("Authenticating using password...");
            return true; // For simplicity, assume authentication always succ
        }
    }
    // FingerprintAuthenticator.cs
    public class FingerprintAuthenticator : IAuthenticator
    {
        public bool Authenticate()
        {
            // Logic for fingerprint authentication
            Console.WriteLine("Authenticating using fingerprint...");
            return true;
        }
    }
    // FaceRecognitionAuthenticator.cs
    public class FaceRecognitionAuthenticator : IAuthenticator
    {
        public bool Authenticate()
        {
```

```

        // Logic for face recognition authentication
        Console.WriteLine("Authenticating using face recognition...");
        return true;
    }
}
//Step 3: Implement the authentication process using the interfaces.
public class AuthService
{
    private IAuthenticator _authenticator;
    public AuthService(IAuthenticator authenticator)
    {
        _authenticator = authenticator;
    }
    public void AuthenticateUser()
    {
        if (_authenticator.Authenticate())
        {
            Console.WriteLine("Authentication successful!");
        }
        else
        {
            Console.WriteLine("Authentication failed.");
        }
    }
}
//Step 4: Testing the application.
class Program
{
    static void Main(string[] args)
    {
        AuthService passwordService = new AuthService(new PasswordAuthenti
passwordService.AuthenticateUser();
        AuthService fingerprintService = new AuthService(new
FingerprintAuthenticator());
        fingerprintService.AuthenticateUser();
        AuthService faceService = new AuthService(new
FaceRecognitionAuthenticator());
        faceService.AuthenticateUser();
        Console.ReadKey();
    }
}
}

```

The IAuthenticator interface offers a consistent way to implement different authentication methods. The main AuthService class remains decoupled from the specifics of each method. This architecture makes it easy to add or change authentication methods in the future without affecting the core logic of the AuthService.

Advantages and Disadvantages of Interfaces in C#

Interfaces are a crucial aspect of object-oriented design and provide several benefits when used correctly. However, like any tool or concept, they have advantages and disadvantages.

Advantages of Interfaces in C#:

- **Flexibility and Extensibility:** By defining functionality as an interface, you can easily extend or change the actual implementations without affecting the clients that rely on the interface.
- **Multiple Inheritance:** While C# doesn't support multiple inheritance for classes (meaning a class cannot inherit from more than one class), it does allow a class to implement multiple interfaces. This can be used to simulate multiple inheritance.
- **Decoupling:** Interfaces decouple the specification of a method or property from its actual implementation. This can make systems easier to refactor, evolve, and swap out implementations.
- **Enforced Contract:** Implementing an interface ensures a class adheres to a particular contract. If a class does not provide implementations for all methods of an interface, the compiler will throw an error.
- **Polymorphism:** You can use interfaces to achieve polymorphism. For example, any method that accepts an argument of an interface type can work with any object that implements that interface.
- **Testability:** Interfaces allow for easier unit testing. Mock objects can be created based on interfaces to isolate and test specific components of an application without involving their dependencies.

Disadvantages of Interfaces in C#:

- **Overhead:** Introducing interfaces adds an additional layer of abstraction, which can lead to increased complexity in design and implementation.
- **Versioning Issues:** If you need to add a new method or property to an interface and maintain a library used by others, it can break existing implementations. (Though starting with C# 8, interfaces can have default implementations for methods, mitigating this issue to an extent.)
- **Loss of Clarity:** If overused or used improperly, interfaces can sometimes confuse or obfuscate the intended design of a system, especially if many classes implement multiple interfaces.
- **No Implementation Details:** Interfaces cannot contain any actual implementation (except for default methods starting in C# 8), meaning all derived classes must provide the implementation. This can sometimes lead to repetitive code if many classes have a similar or identical implementation for interface methods.
- **Increased Hierarchy:** If a system has too many interfaces, especially ones with only a single method or property, it can lead to a deep hierarchy, which can be hard to navigate and understand.

When to use Interfaces in C#?

Using interfaces appropriately can be instrumental in crafting maintainable, flexible, and scalable software. In C#, you'd typically consider using interfaces in the following scenarios:

- **Multiple Implementations:** The exact implementations will differ when you expect multiple classes to implement a specific behavior or set of behaviors. An interface can define this common behavior. Example: Different logging mechanisms (ConsoleLogger, FileLogger, DatabaseLogger, etc.) can all implement an ILogger interface.
- **Decoupling:** If you want to decouple your classes and their implementations. This can lead to more modular code, making it easier to swap, test, or extend functionality without major refactoring. Example: Dependency injection in ASP.NET Core heavily relies on interfaces to provide and swap out services.
- **Abstraction:** When you want to hide the details of a class and only show its essential features, using an interface can be beneficial. Example: In a graphics system, shapes like Circle, Rectangle, and

Triangle can implement an IDrawable interface.

- **Achieving Polymorphism:** Interfaces allow different classes to be treated as instances of the same type, enabling polymorphism. This can simplify code by allowing different classes to be used interchangeably based on their shared interface. Example: Any class implementing the IComparable interface can be sorted using generic algorithms.
- **Contract Enforcement:** When you want to ensure that certain classes implement specific methods or properties, an interface acts as a contract that enforces this. You'll get a compile-time error if a class doesn't fulfill the contract. Example: Implementing the IEnumerable interface ensures a class provides an enumerator for its elements.
- **Plugin Architecture:** If you're designing a system where third parties might extend it with plugins or additional functionalities, interfaces can act as a blueprint for these extensions. Example: A media player that allows third-party codec plugins might have an ICodec interface.
- **Simulating Multiple Inheritance:** C# does not support multiple inheritance for classes, but a class can implement multiple interfaces. This can be useful if you want a class to inherit behaviors from multiple sources. Example: A Smartphone class can implement both ICamera and IPhone interfaces.
- **Standardizing APIs:** When creating libraries or frameworks for others to use, interfaces can help standardize the API, making it clear to users how to interact with your library. Example: The IDisposable interface in .NET provides a standard way to release unmanaged resources.

In the next article, I will discuss the [Multiple Real-time Examples of Abstract Classes in C#](#). In this article, I explain **Real-Time Examples of Interfaces in C#**. I hope you enjoy this Interface Real-Time Examples using the C# article.